

PENGENALAN GET X

Apa itu Get X

Flutter punya banyak library state management untuk dipilih, seperti Provider, BLoC, Redux, Riverpod, GetX dsb. Nah, dalam artikel kali ini, kita akan berkenalan dengan GetX, yang merupakan salah satu library state management paling populer di Flutter pada saat artikel ini ditulis.

GetX berbeda dengan state management lainnya karena GetX bisa disebut sebagai microframework di Flutter. Bisa disebut demikian karena selain menawarkan fitur state management, pada package GetX juga sudah tersedia Dependency Injection, Routing dan juga menyediakan banyak fitur tambahan yang berguna untuk pengembangan aplikasi Flutter. Seperti misalnya, fitur GetUtils yang menyediakan utilitas umum seperti konversi waktu dan tanggal, manipulasi string, dan validasi input user. GetX juga punya fitur GetStorage untuk menyimpan data secara lokal dengan mudah dan tahan lama. Dengan fitur-fitur tambahan ini, GetX sangat cocok digunakan dalam proyek kecil sampai menengah yang butuh solusi cepat dan efisien.

GetX sendiri dibuat dengan berpatokan pada tiga prinsip yaitu:

- **Performance:** GetX berfokus agar ia tidak boros resource sehingga menghasilkan performance yang baik. Oleh karena itu GetX tidak menggunakan Stream atau ChangeNotifier.
- **Productivity:** GetX dirancang untuk mempermudah proses development. GetX menggunakan sintaks yang mudah dan intuitif sehingga mudah dipahami & hemat waktu karena hampir tidak ada boilerplate code yang harus kita tulis. Kita juga tidak perlu khawatir tentang memory management karena controller pada GetX secara default akan dihapus dari memori, kalo kita butuh controller-nya persist, kita cukup override settingnya saja. Dependency Injection juga dilakukan secara lazy by default, sehingga kita tidak perlu meng-inisialisasi dependency secara manual.
- **Organization:** GetX secara default memudahkan kita untuk mengadopsi prinsip separation of concern . GetX memfasilitasi pemisahan View, Presentation logic, Business logic, Dependency injection, dan Navigation. Hal ini karena pada GetX kita tidak perlu context sehingga view yang kita buat tidak bergantung pada widget tree (tidak perlu akses Controller/Bloc/Provider menggunakan context) & tidak memerlukan dependency injection tambahan, sehingga code kita benar-benar decoupled satu sama lain.

Sebenarnya masih ada beberapa kelebihan lain dari GetX, mulai dari adanya Get CLI sampai VS Code extension yang bisa mempercepat dan mempermudah proses development kita tapi kita akan bahas itu nanti, tapi jika teman-teman penasaran, teman-teman bisa baca lebih lanjut di sini

Selanjutnya kita bakal bahas sedikit mengenai dua style yang bisa kita pilih dalam menggunakan GetX state management.

1. Reactive Style

Pemrograman reaktif adalah cara elegan buat menggambarkan gimana aplikasi atau widget berperilaku berdasarkan perubahan dalam state-nya. Dengan GetX, kita pake dua kelas, Rx dan Obx, buat menerapkan pemrograman reaktif. Kelas Rx memungkinkan kita buat definisi variabel reaktif atau observables. Widget Obx adalah widget khusus yang subscribed ke sebuah observable dan rebuild dirinya setiap kali observable berubah.

Contoh pengelolaan reactive state management di GetX:

```
import 'package:get/get.dart';

class CounterController extends GetxController {
  RxInt counter = 0.obs;

  void increment() => counter.value++;
}
```

Dalam contoh di atas, kita punya kelas CounterController yang extend kelas GetXController. Kita juga punya reactive variable counter dengan tipe RxInt dengan nilai awal 0. Metode increment dipake buat nambahin nilai counter. Dan berikut adalah contoh gimana pake widget Obx buat nampilin variabel counter:

```
class CounterPage extends StatelessWidget {
  final CounterController controller = Get.put(CounterController());

  @override
  Widget build(BuildContext context) {
    print("Build method called");
    return Scaffold(
      appBar: AppBar(title: Text('Counter')),
      body: Center(
        child: Obx(() => Text('Count: ${controller.counter}')),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: controller.increment,
        child: Icon(Icons.add),
      ),
    );
  }
}
```

Dalam contoh di atas, kita punya widget CounterPage di mana di dalamnya ada widget Obx yang membungkus widget Text yang berfungsi untuk nampilin variabel counter. Setiap kali nilai counter berubah, widget Obx akan me-rebuild UI yang terbungkus di dalamnya dan nampilin nilai baru dari counter. Hanya widget yang berisikan variable Rx dari controller-lah yang akan di-rebuild, karena itulah penggunaan GetX meningkatkan performance aplikasi kita sebab GetX meminimalisir UI yang di-rebuild.

2. Simple Style

Style ini mirip seperti saat kita menggunakan `ChangeNotifierProvider` atau `setState`. Dalam style ini kita menggunakan widget `GetBuilder` yang bertugas untuk observe perubahan state & rebuild widget yang mengandung state yang berubah. Contoh :

```
import 'package:get/get.dart';

class CounterController extends GetxController {
  var counter = 0;

  void increment() {
    counter++;
    update();
  }
}
```

Dalam contoh di atas, kita punya kelas `CounterController` yang extend `GetXController`. Kita juga punya variabel `counter` dengan nilai awal 0. Metode `increment` dipake buat nambahin nilai `counter` dan manggil metode `update()` yang fungsinya mirip seperti `notifyListener()` pada `ChangeNotifier` atau `setState()` pada `StatefulWidget`. Dan berikut adalah contoh gimana pake widget `GetBuilder` buat nampilin variabel `counter`:

```
class CounterPage extends StatelessWidget {
  final CounterController controller = Get.put(CounterController());

  @override
  Widget build(BuildContext context) {
    print("Build method called");
    return Scaffold(
      appBar: AppBar(title: Text('Counter')),
      body: Center(
        child: GetBuilder<CounterController>(
          builder: (controller) {
            return Text('Count: ${controller.counter}');
          },
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: controller.increment,
        child: Icon(Icons.add),
      ),
    );
  }
}
```

Dalam contoh di atas, kita punya widget CounterPage yang pake widget GetBuilder buat nampilin variabel counter. Setiap kali nilai counter berubah, widget GetBuilder akan rebuild widget yang terbungkus di dalamnya & mengandung state yang berubah.

- Instalasi Package GetX

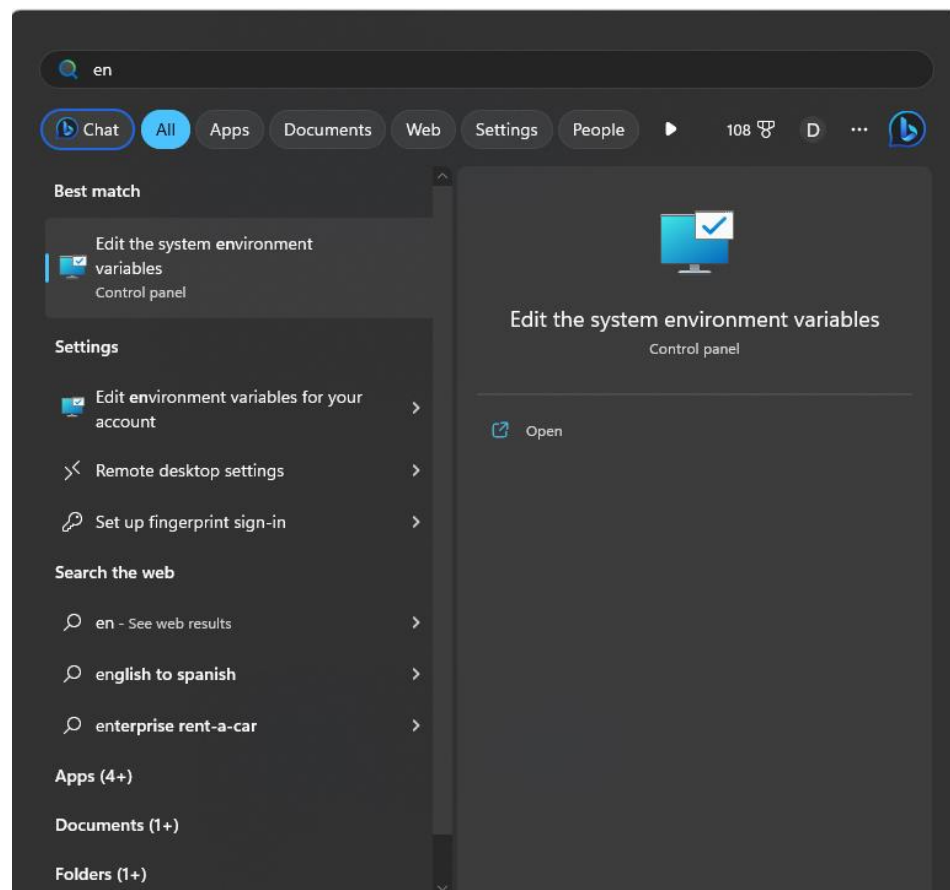
Sebelum membuat project dengan menggunakan GetX, kita diwajibkan untuk instalasi beberapa package yang akan digunakan dalam project, salah satunya adalah get cli. Untuk melakukan install Get CLI bisa menuju ke link berikut ini : https://pub.dev/packages/get_cli/install .

1. Instalasi package Get X dengan menggunakan cli:

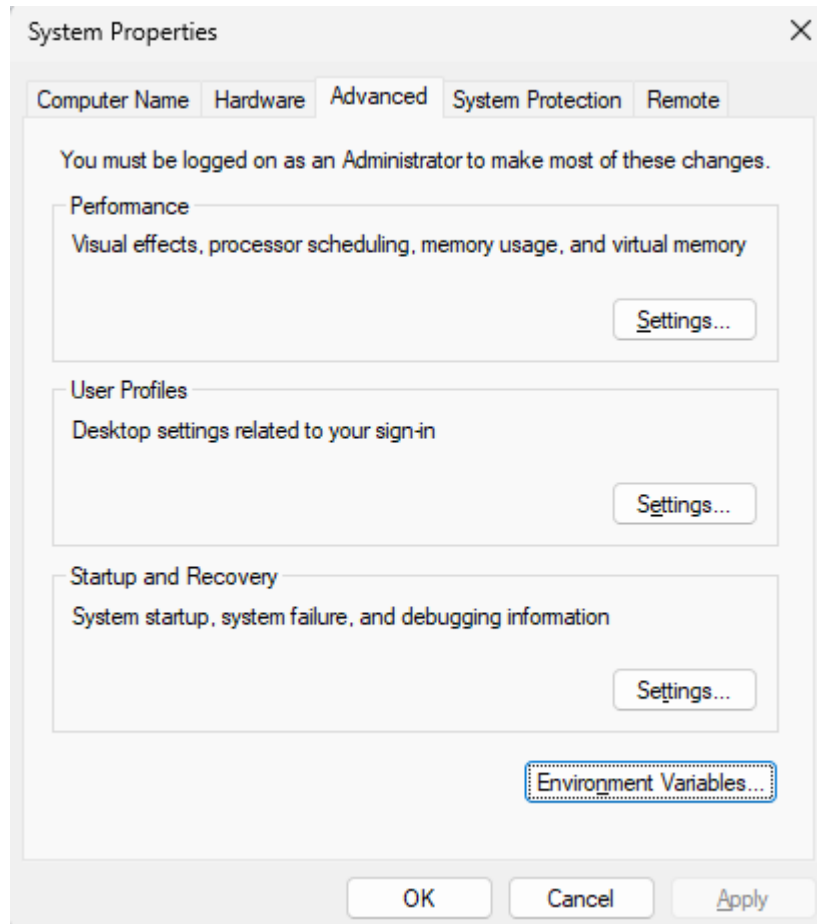
```
flutter pub global activate get_cli
```

2. Setelah selesai kita perlu menambahkan path ke *environment variables* sehingga bisa dipanggil dimana saja. Untuk itu langkah-langkah nya adalah sebagai berikut ini

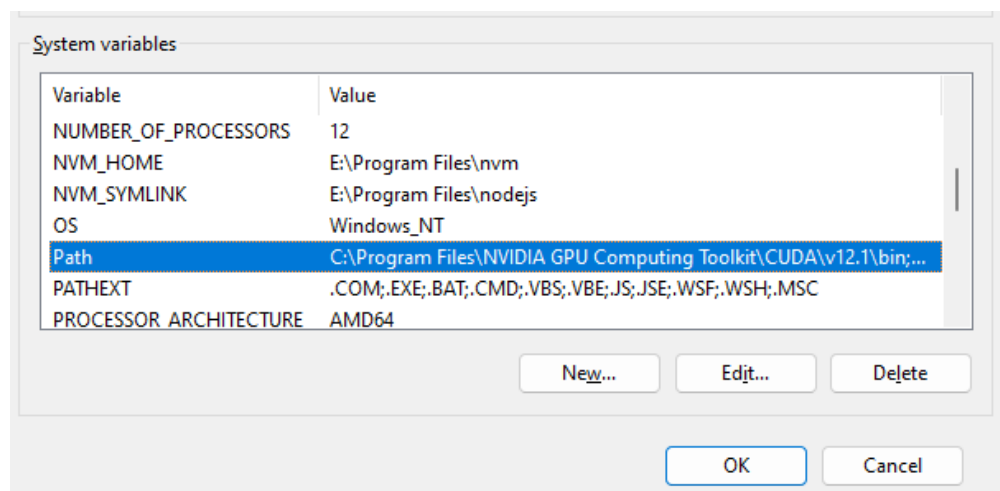
3.
1. Buka Environment Variable dengan cara masuk ke menu 'Edit the system environment variables'.



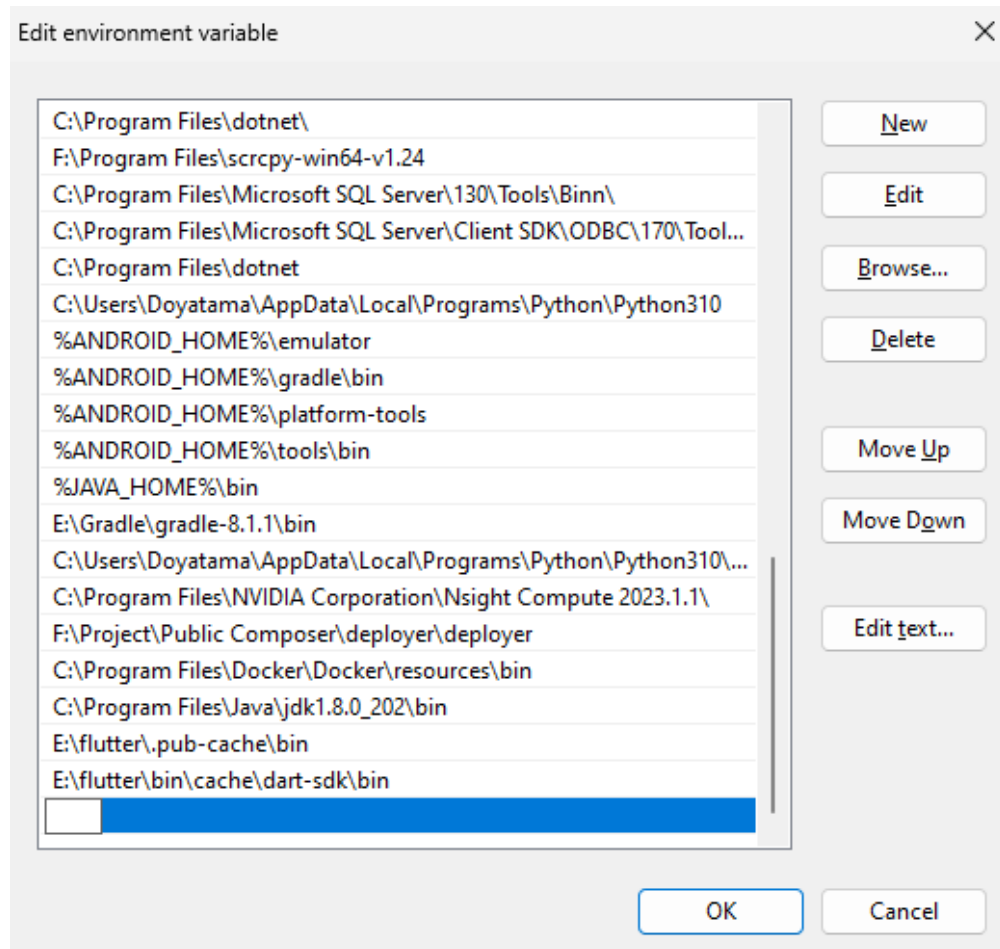
- b. Pilih "environment variables" pada tab hardware



- c. Pada tabs System Variable, cari variable 'Path' lalu klik edit



- d. Klik New pada Edit Environment Variable



- e. Tambahkan path dari .pub-cache yang lokasinya ada di folder master dari flutter, berikut contohnya:

E:\flutter\.pub-cache\bin

- f. Jika sudah selesai, saatnya dicoba di cmd dengan cara mengetikkan 'get'. Jika berhasil maka akan menampilkan tampilan seperti berikut ini.

```
CA\WINDOWS\system32\cmd.exe
  locales: Generate translation file from json files
  model: generate Class model from json
  help: Show this help
  init: generate the chosen structure on an existing project:
  install: Use to install a package in your project (dependencies):
  remove: Use to remove a package in your project (dependencies):
  sort: Sort imports and format dart files
  update: To update GET_CLI
  --version: Shows the current CLI version'

There's an update available! Current installed version: 1.8.1

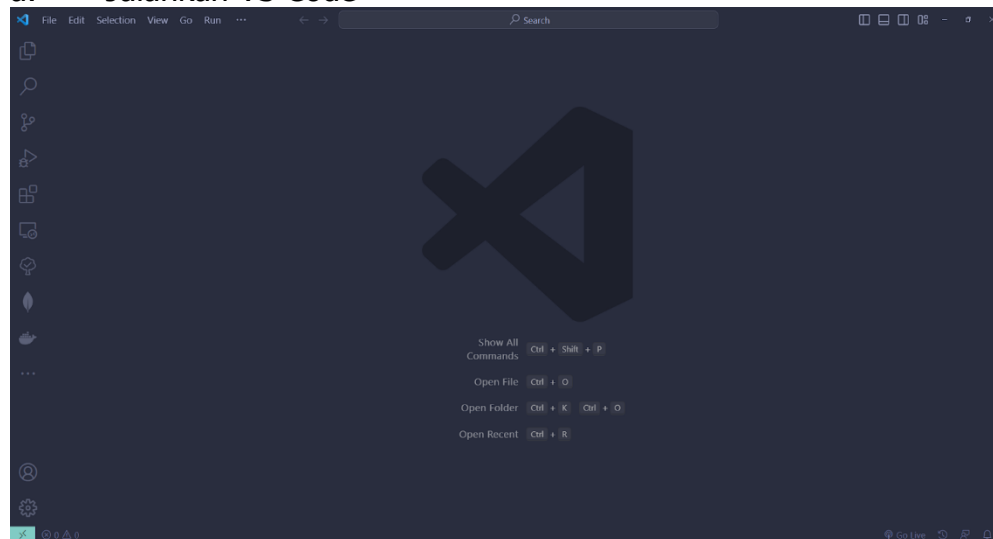
GET CLI

New version available: 1.8.4 Please, run: get update

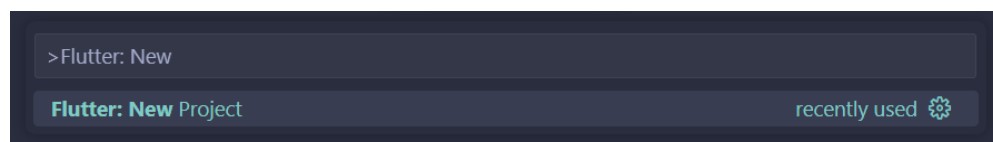
Time: 552 Milliseconds

C:\Users\Doyatama>
```

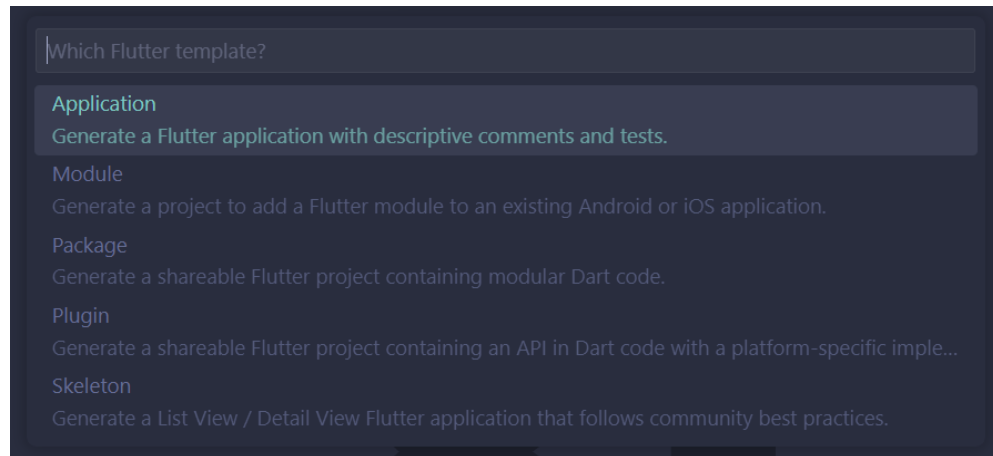
1. Jika sudah saat nya membuat project untuk CRUD menggunakan GetX.
 - a. Jalankan VS Code



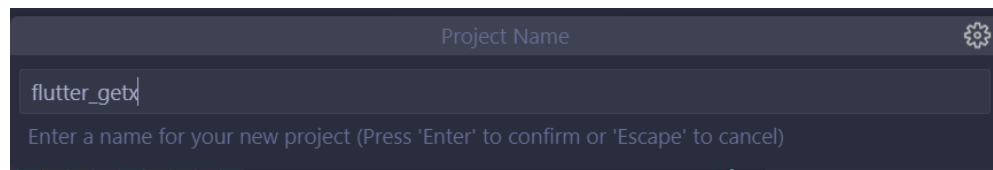
- b. Ketik CTRL + SHIFT + P kemudian ketik dan pilih 'Flutter: New Project'.



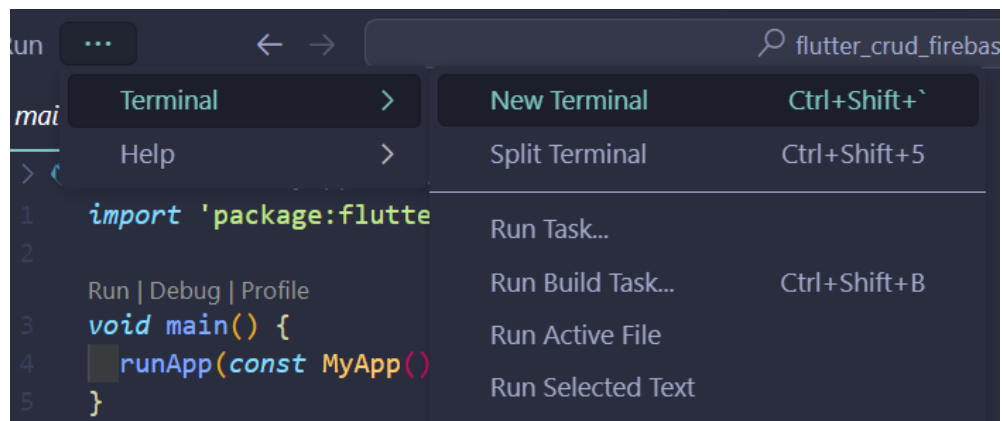
- c. Pilih Application, lalu pilih lokasi penyimpanan project yang akan digunakan.



- d. Berikan nama project, disini penulis memberikan nama 'flutter_getx'.



- e. Selanjutnya jalankan perintah 'get init' pada terminal VS Code



- f. Selanjutnya pilih 'GetX Pattern' dan pilih 'Yes', maka akan menjadi seperti berikut ini:

```
? Which architecture do you want to use? (Use arrow keys)
> GetX Pattern (by Kauê)
  CLEAN (by Arktekko)
? Your lib folder is not empty. Are you sure you want to overwrite your application?
WARNING: This action is irreversible (Use arrow keys)
> Yes!
No
✓ 'Package: get installed!
✓ File: main.dart created successfully at path: lib\\main.dart
✓ File: home_controller.dart created successfully at path: lib\\app\\modules\\home\\controllers\\home_controller.dart
✓ File: home_view.dart created successfully at path: lib\\app\\modules\\home\\views\\home_view.dart
✓ File: home_binding.dart created successfully at path: lib\\app\\modules\\home\\bindings\\home_binding.dart
✓ File: app_routes.dart created successfully at path: lib\\app\\routes\\app_routes.dart
✓ File: app_pages.dart created successfully at path: lib\\app\\routes\\app_pages.dart
✓ home route created successfully.
✓ Home page created successfully.
✓ GetX Pattern structure successfully generated.

Running 'flutter pub get' ...
```

- g. Setelah selesai selanjutnya bisa menonaktifkan kode berikut ini pada file 'test/widget_test.dart'

```
Run | Debug
testWidgets('Counter increments smoke test', (W
// Build our app and trigger a frame.
// await tester.pumpWidget(const MyApp());
```

Jika sudah selesai melakukan get init, maka anda bisa membuat halaman dengan sangat mudah, yaitu dengan menggunakan perintah seperti berikut ini : (misalkan ingin membuat halaman login).

```
get create page:login
```

Untuk berpindah layer dari layer 1 ke lainnya bisa menggunakan perintah `Get.toNamed(Routes.LOGIN)` pada `InkWell`. Berikut contohnya :

```
InkWell(
  onTap: () => {
    Get.toNamed(
      Routes.LOGIN,
    ),
  },
  child: Text('pindah'),
```

)

Tugas

Buatkan minimal 5 Halaman dan bisa dilakukan pindah pindah halaman serta sudah ada beberapa widget di dalamnya.